

I/O Fundamentals:
Handshaking,
Buffering
I/O Mapping –
Program Controlled
I/O



Introductions

- ◆ I/O plays a crucial role in any modern computer system.
- ◆ I/O organization of a computer is a function of size of the computer and the device connected to it.
- ◆ In other words, amounts of hardware computer processes to communicate with number of peripheral units, differentiate between small and large system.

I/O Subsystem

- ◆ The input-output subsystem of a computer, referred as I/O, provides an efficient mode of communication between the central system and outside environment.
- ◆ Data and programs must be entered into the computer memory for processing and result of computations must be recorded or displayed for the user.

Input – Output Interface

- ◆ Peripherals are electromechanical and electromagnetic devices and manner of operation is different from that of CPU which is electronic component.
- ◆ Data transfer rate of peripherals is slower than that of CPU. So some synchronization mechanism may be needed.
- ◆ Data codes and formats in peripherals differ from the word format in CPU and memory.
- ◆ Operating modes of peripherals are different from each other and each must be controlled so as not to disturb other.

Handshaking

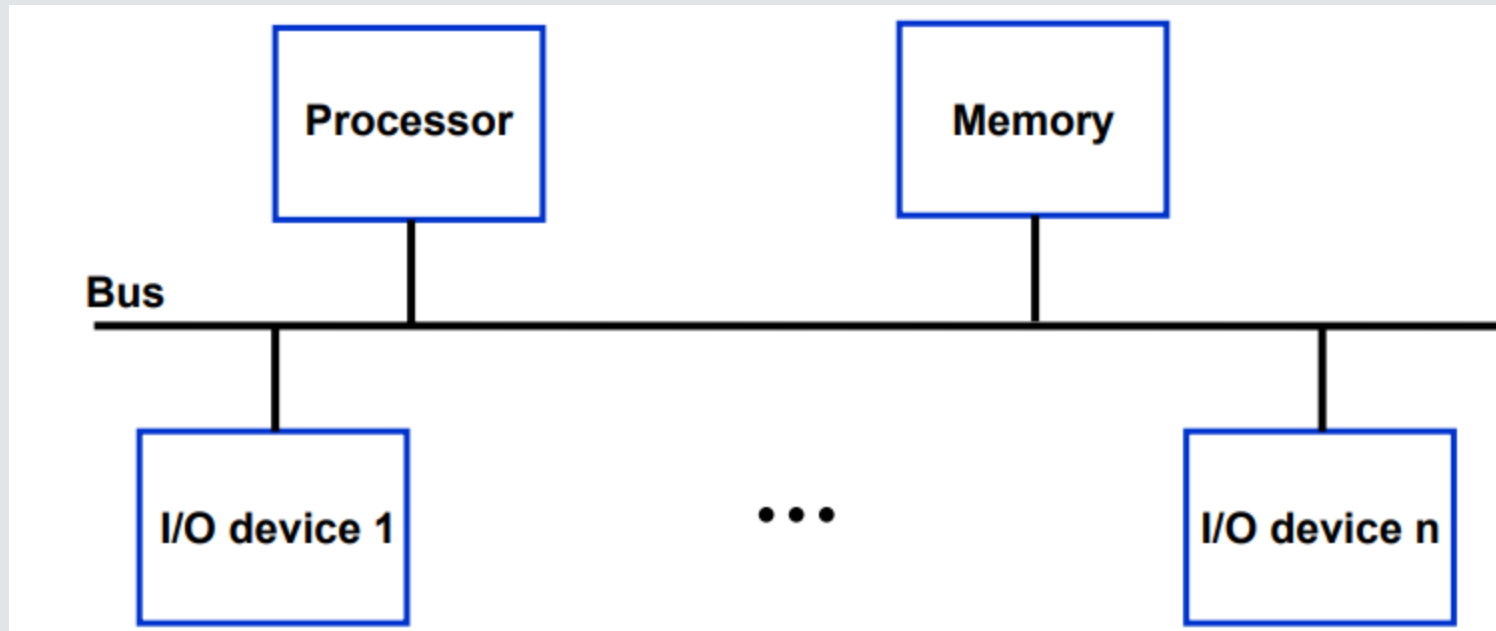
- ◆ Handshaking is a I/O control method to synchronize I/O devices with the microprocessor.
- ◆ Supposing that we have a printer connected to a system. The printer can print 100 characters/second, but the microprocessor can send much more information to the printer at the same time.
 - ◆ That's why, just when the printer gets it enough data to print it places a logic 1 signal at its Busy pin, indicating that it is busy in printing.
 - ◆ The microprocessor now tests the busy bit to decide if the printer is busy or not.
 - ◆ When the printer will become free it will change the busy bit and the microprocessor will again send enough amounts of data to be printed.
 - ◆ This process of interrogating the printer is called handshaking.

I/O Buffering

- ◆ A buffer is a memory area that stores data being transferred between two devices or between a device and an application.
- ◆ Buffering is a mechanism that improves the throughput of input and output operations.
- ◆ No direct transfer from (to) user to (from) I/O device
- ◆ Used to smooth peaks in I/O traffic
- ◆ Particularly useful for dealing with scenarios where there is a difference in data transfer rates - linking fast devices to slow ones for instance
- ◆ Also useful for storing data temporarily ...
- ◆ *OUTPUT* : User's output data transferred to OS output buffer. User continues operation, and is only suspended when the buffer becomes full
- ◆ *INPUT* : OS reads ahead for user. Read normally satisfied from buffer. User is blocked if buffer is empty

Accessing I/O Devices

- Single-bus structure
 - The bus enables all the devices connected to it to exchange information.
 - Typically, the bus consists of three sets of lines used to carry address, data, and control signals.
 - Each I/O device is assigned a unique set of addresses



I/O Mapping

- Memory mapped I/O
 - Devices and memory share an address space
 - I/O looks just like memory read/write
 - No special commands for I/O
 - Large selection of memory access commands available
- Isolated I/O
 - Separate address spaces
 - Need I/O or memory select lines
 - Special commands for I/O
 - Limited set

I/O Techniques

- Programmed
- Interrupt driven
- Direct Memory Access (DMA)

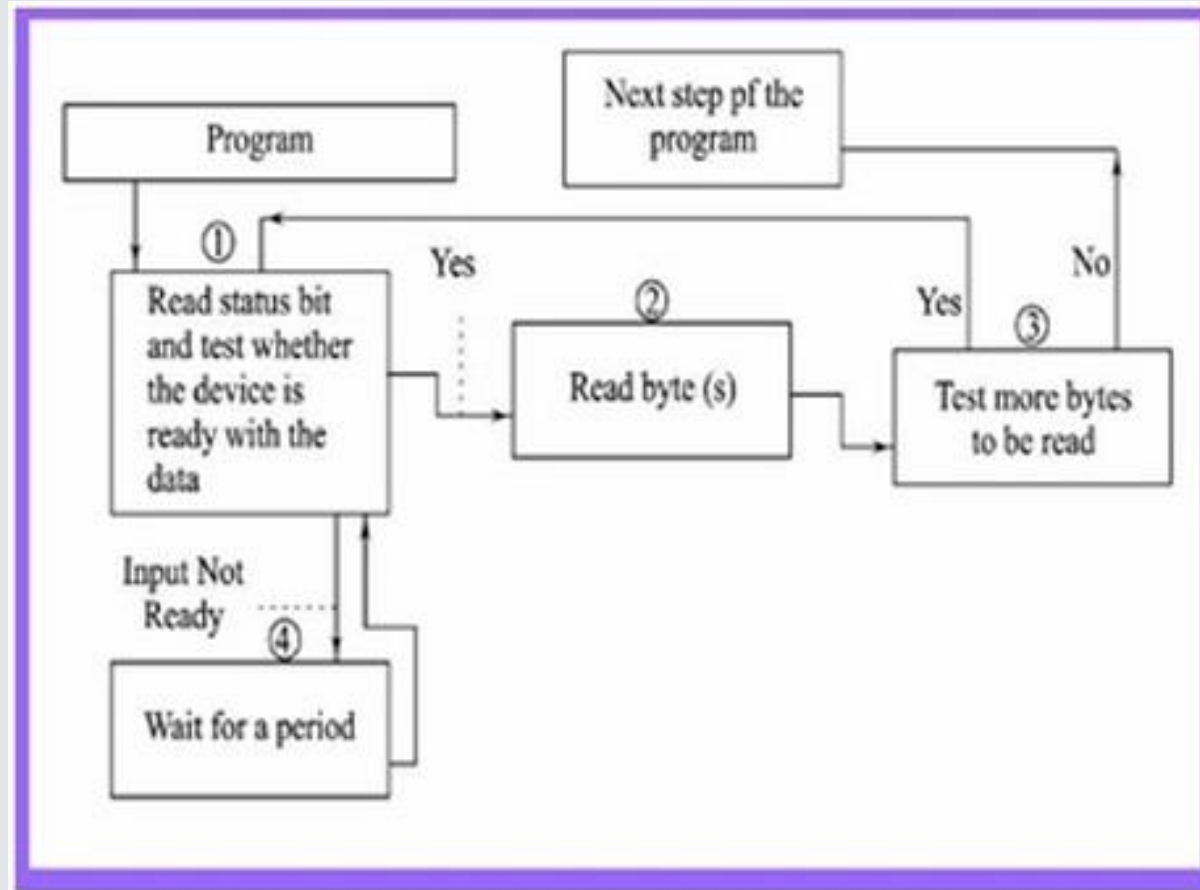
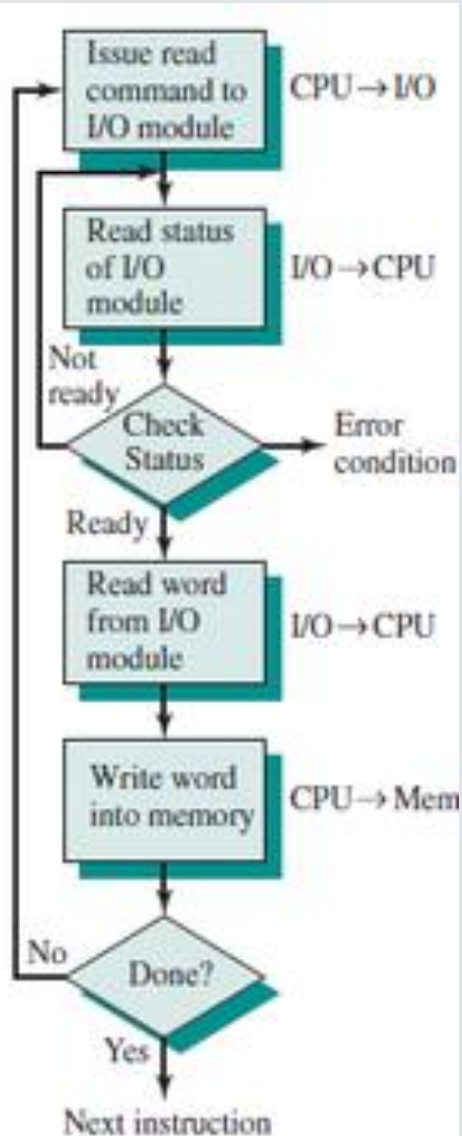
Program-Controlled I/O

- ❖ The programmed I/O was the most simple type of I/O technique for the exchanges of data or any types of communication between the processor and the external devices.
- ❖ With programmed I/O, data are exchanged between the processor and the I/O module.
- ❖ The processor executes a program that gives it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data.
- ❖ When the processor issues a command to the I/O module, it must wait until the I/O operation is complete.

Program-Controlled I/O

- ◆ If the processor is faster than the I/O module, this is wasteful of processor time. The overall operation of the programmed I/O can be summarized as follows:
 - ◆ *The processor is executing a program and encounters an instruction relating to I/O operation.*
 - ◆ *The processor then executes that instruction by issuing a command to the appropriate I/O module.*
 - ◆ *The I/O module will perform the requested action based on the I/O command issued by the processor (READ/WRITE) and set the appropriate bits in the I/O status register.*
 - ◆ *The processor will periodically check the status of the I/O module until it finds that the operation is complete.*

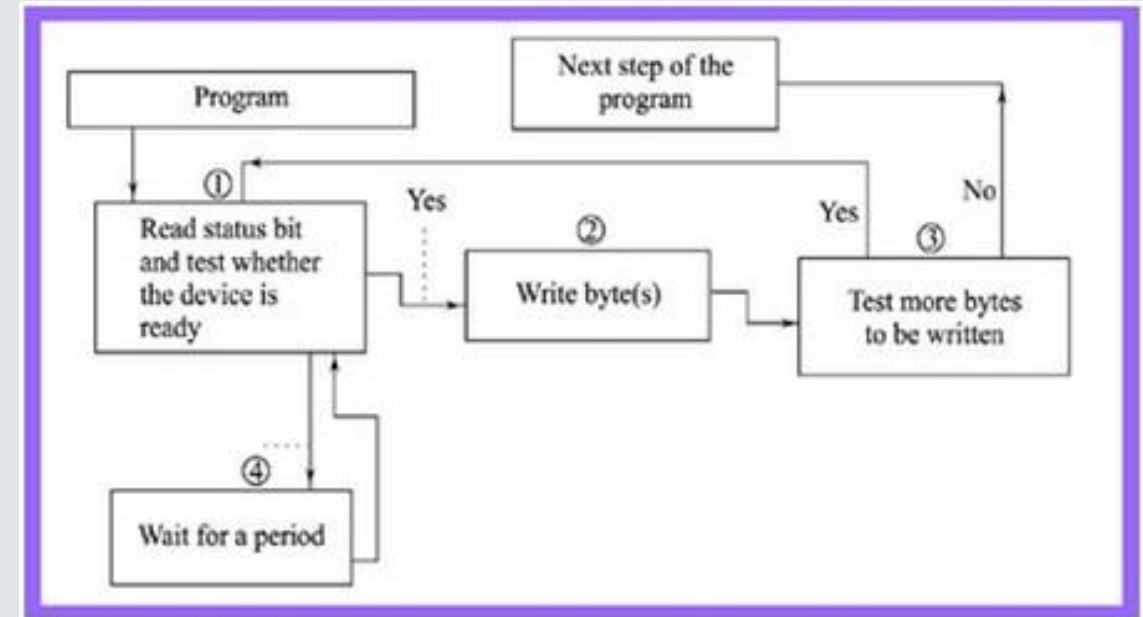
Programmed I/O Mode Input Data Transfer



- ❖ Each input is read after first testing whether the device is ready with the input (a state reflected by a bit in a status register).
- ❖ The program waits for the ready status by repeatedly testing the status bit and till all targeted bytes are read from the input device.
- ❖ The program is in busy (non-waiting) state only after the device gets ready else in wait state.

Programmed I/O Mode Output Data Transfer

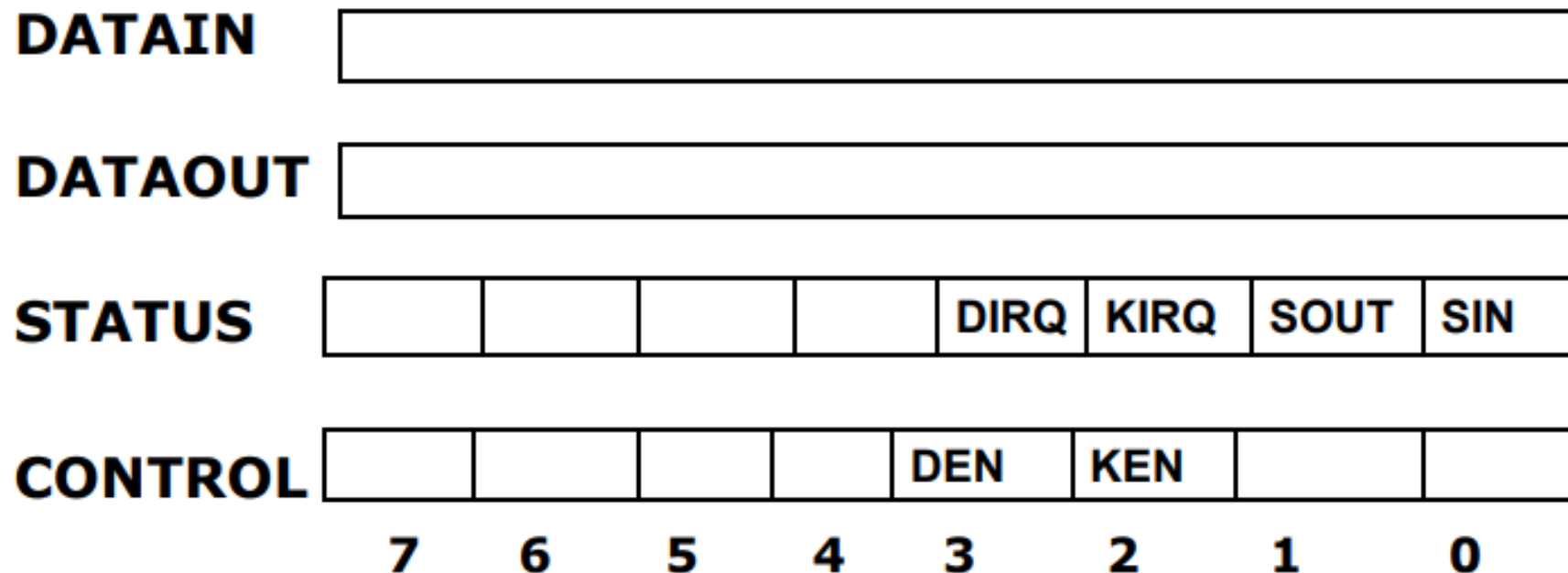
- ◆ Each output written after first testing whether the device is ready to accept the byte at its output register or output buffer is empty.
- ◆ The program waits for the ready status by repeatedly testing the status bit(s) and till all the targeted bytes are written to the device.
- ◆ The program in busy (non-waiting) state only after the device gets ready else wait state.



- ❖ In this case, use dedicated I/O instructions in the processor. These I/O instructions can specify both the device number and the command word (or the location of the command word in memory).
- ❖ The processor communicates the device address via a set of wires normally included as part of the I/O bus. The actual command can be transmitted over the data lines in the bus. (example - Intel IA-32).
- ❖ By making the I/O instructions illegal to execute when not in kernel or supervisor mode, user programs can be prevented from accessing the devices directly.
- ❖ The process of periodically checking status bits to see if it is time for the next I/O operation, is called polling. Polling is the simplest way for an I/O device to communicate with the processor.
- ❖ The I/O device simply puts the information in a Status register, and the processor must come and get the information. The processor is totally in control and does all the work.

I/O Commands

- ◆ To execute an I/O-related instruction, the processor issues an address, specifying the particular I/O module and external device, and an I/O command. There are four types of I/O commands that an I/O module may receive when it is addressed by a processor:
 - ◆ **Control:** Used to activate a peripheral and tell it what to do. For example, a magnetic-tape unit may be instructed to rewind or to move forward one record. These commands are tailored to the particular type of peripheral device.
 - ◆ **Test:** Used to test various status conditions associated with an I/O module and its peripherals. The processor will want to know that the peripheral of interest is powered on and available for use. It will also want to know if the most recent I/O operation is completed and if any errors occurred.
 - ◆ **Read:** Causes the I/O module to obtain an item of data from the peripheral and place it in an internal buffer. The processor can then obtain the data item by requesting that the I/O module place it on the data bus.
 - ◆ **Write:** Causes the I/O module to take an item of data (byte or word) from the data bus and subsequently transmit that data item to the peripheral



I/O operation involving keyboard and display devices

Registers: DATAIN, DATAOUT, STATUS, CONTROL

Flags: SIN, SOUT - Provides status information for keyboard and display unit

KIRQ, DIRQ – Keyboard, Display Interrupt request bits

DEN, KEN –Keyboard, Display Enable bits

A ISA program to read one line from the keyboard, store it in memory buffer and echo it back to the display memory buffer , and echo it back to the display

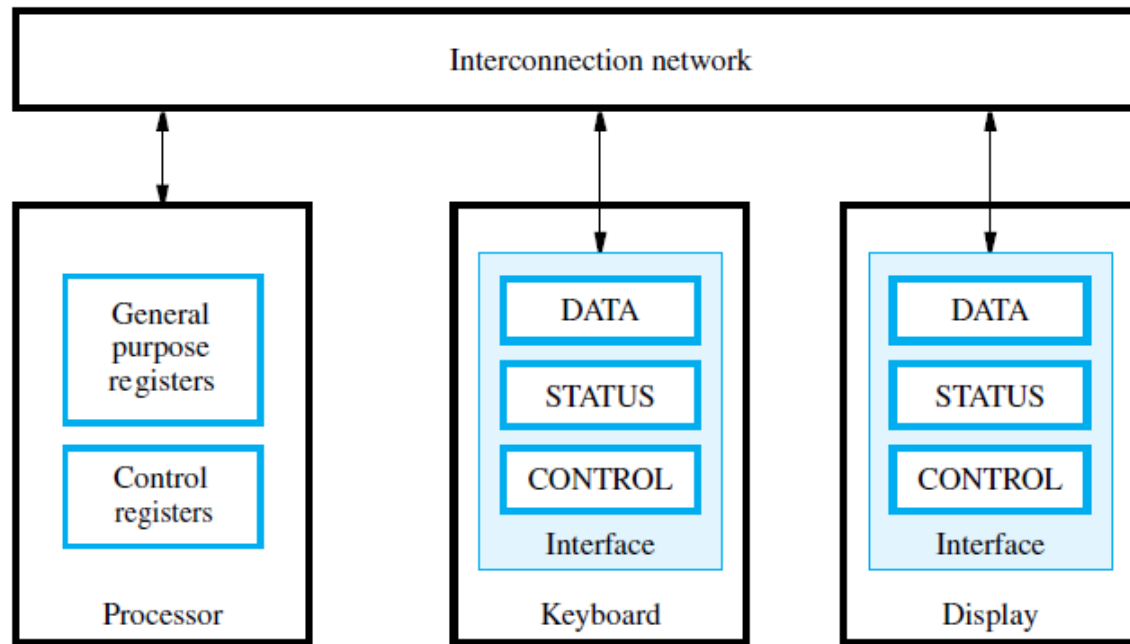


Figure 3.2 The connection for processor, keyboard, and display.

```
Move    #LINE,R0           Initialize memory pointer.
WAITK   TestBit    #0,STATUS Test SIN.
        Branch=0   WAITK    Wait for character to be entered.
        Move      DATAIN,R1 Read character.
WAITD   TestBit    #1,STATUS Test SOUT.
        Branch=0   WAITD    Wait for display to become ready.
        Move      R1,DATAOUT Send character to display.
        Move      R1,(R0)+ Store character and advance pointer.
        Compare   #$0D,R1   Check if Carriage Return.
        Branch≠0 WAITK    If not, get another character.
        Move      #$0A,DATAOUT Otherwise, send Line Feed.
        Call      PROCESS  Call a subroutine to process the
                                the input line.
```

- ❖ The disadvantage of polling is that it can waste a lot of processor time because processors are so much faster than I/O devices.
- ❖ The processor may read the Status register many times, only to find that the device has not yet completed a comparatively slow I/O operation, or that the mouse has not budged since the last time it was polled.
- ❖ When the device completes an operation, we must still read the status to determine whether it (I/O) was successful.
- ❖ Overhead in a polling interface lead to the invention of interrupts to notify the processor when an I/O device requires attention from the processor.
- ❖ Interrupt -driven I/O employs I/O interrupts to indicate to the processor that an I/O device needs attention.
- ❖ When a device wants to notify the processor that it has completed some operation or needs attention, it causes the processor to be interrupted.

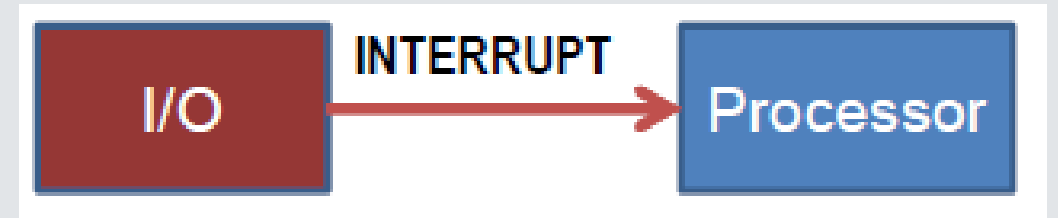
Interrupts



Interrupts

- ◆ In the programmed control led IO, the processor has to continuously monitor the devices for any input/output data
- ◆ This process over burden the process to monitor the IO devices at a regular an intervals
- ◆ An alternate approach to perform IO operation i.e. when the IO devices are ready to transfer or receive the data then it can notify the processor by means of interrupts
- ◆ Interrupt is a hardware signal send by the peripherals to the processor
- ◆ The processor no longer has to monitor the IO peripherals
- ◆ How the processor react for an Interrupt?
 - ◆ In abstract, The processor pause its ongoing process execution, It acknowledges the interrupting device and executes the Interrupt Service Routine(ISR) ,after the execution of ISR the processor resumes the execution of the ongoing process.

Interrupts

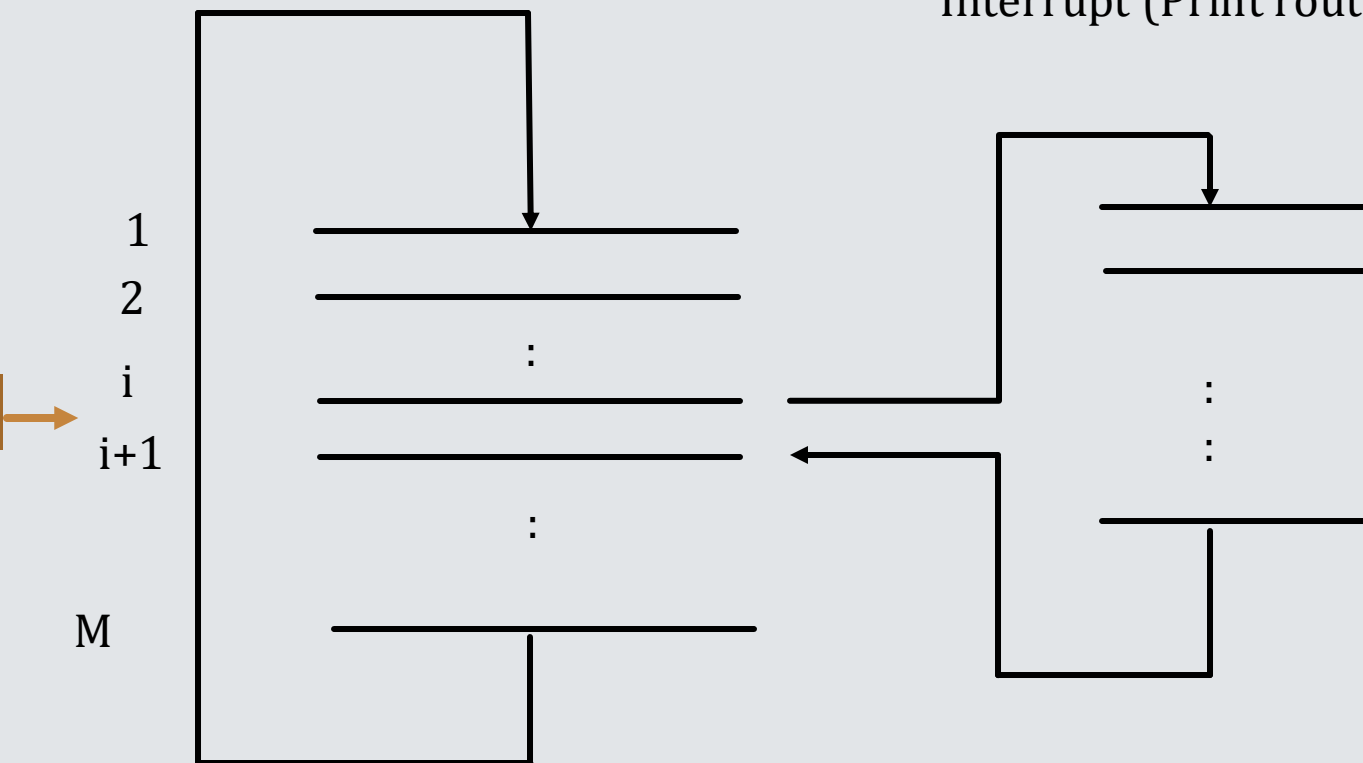


- ◆ When I/O Device is ready, it sends the INTERRUPT signal to processor via a dedicated controller line.
- ◆ Using interrupt we are ideally eliminating WAIT period.
- ◆ In response to the interrupt, the processor executes the Interrupt Service Routine (ISR).
- ◆ All the registers, flags, program counter values are saved by the processor before running ISR.
- ◆ The time required to save status & restore contribute to execution overhead → “Interrupt Latency”.

Main Program

Interrupt (Print routine)

Interrupt occurs
here



- ◆ Interrupt-acknowledge signal - I/O device interface accomplishes this by execution of an instruction in the interrupt-service routine (ISR) that accesses a status or data register in the device interface; implicitly informs the device that its interrupt request has been recognized. IRQ signal is then removed by device.
- ◆ ISR is a sub-routine – may belong to a different user than the one being executed and then halted.
- ◆ The condition code flags and the contents of any registers used by both the interrupted program and the interrupt-service routine are saved and restored.
- ◆ The concept of interrupts is used in operating systems and in many control applications, where processing of certain routines must be accurately timed relative to external events (e.g. real-time processing).

Enabling and Disabling Interrupts

- ◆ Device activates interrupt signal line and waits with this signal activated until processors attends
- ◆ The interrupt signal line is active during execution of ISR and till the device caused interrupt is serviced
- ◆ Necessary to ensure that the active signal does not lead to successive interruptions (level-triggered input) causing the system to fall in infinite loop.
- ◆ Three methods of Controlling Interrupts (single device)
 - ◆ Ignoring interrupt
 - ◆ Disabling interrupts
 - ◆ Special Interrupt request line

◆ Ignoring Interrupts

- ◆ Processor hardware ignores the interrupt request line until the execution of the first instruction of the ISR completed
- ◆ Using an interrupt disable instruction after the first instruction of the ISR – no further interrupts
- ◆ A return from interrupt instruction is completed before further interruptions can occur

◆ Disabling Interrupts

- ◆ Processor automatically disables interrupts before starting the execution of the ISR
- ◆ The processor saves the contents of PC and PS (status register) before performing interrupt disabling.
- ◆ The interrupt-enable is set to 0 – no further interrupts allowed
- ◆ When return from interrupt instruction is executed the contents of the PS are restored from the stack, and the interrupt enable is set to 1

◆ Special Interrupt line

- ◆ Special interrupt request line for which the interrupt handling circuit responds only to the leading edge of the signal
- ◆ Edge –triggered
- ◆ Processor receives only one request regardless of how long the line is activated
- ◆ No separate interrupt disabling instructions

- ◆ The sequence of events involved in handling an interrupt request from a single device.
- ◆ Assuming that interrupts are enabled, the following is a typical scenario:
 1. The device raises an interrupt request.
 2. The processor interrupts the program currently being executed.
 3. Interrupts are disabled by changing the control bits in the PS (except in the case of edge-triggered interrupts).
 4. The device is informed that its request has been recognized, and in response, it deactivates the interrupt request signal.
 5. The action requested by the interrupt is performed by the interrupt-service routine.
 6. Interrupts are enabled and execution of the interrupted program is resumed.

Handling Multiple Devices

- ◆ Multiple devices can initiate interrupts
- ◆ They use the common interrupt request line
- ◆ Techniques are:-
 - ◆ Polling
 - ◆ Vectored Interrupts
 - ◆ Interrupt Nesting
 - ◆ Daisy Chaining

Polling Scheme

- ◆ The IRQ (interrupt request) bit in the status register is set when a device is requesting an interrupt.
- ◆ The Interrupt service routine polls the I/O devices connected to the bus.
- ◆ The first device encountered with the IRQ bit set is serviced and the subroutine is invoked.
- ◆ Easy to implement, but too much time spent on checking the IRQ bits of all devices, though some devices may not be requesting service.

Vectored Interrupts

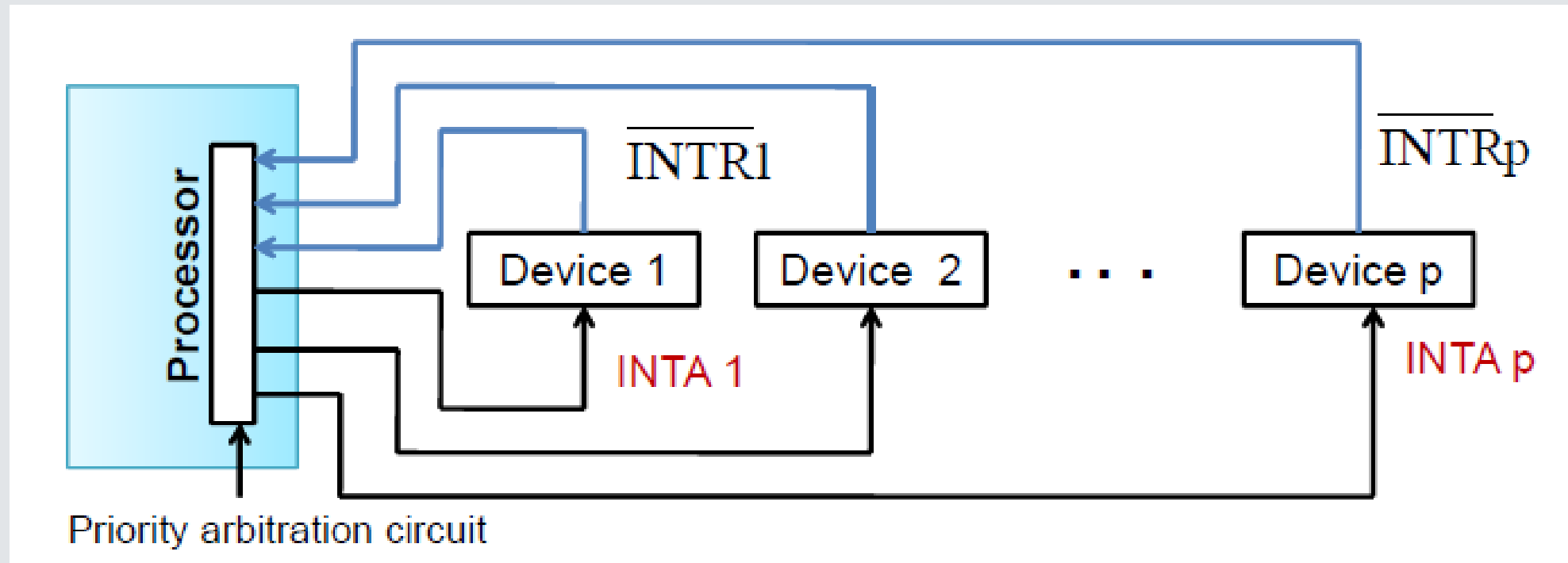
- ◆ Device requesting an interrupt identifies itself directly to the processor
- ◆ The device sends a special code to the processor over the bus.
- ◆ The code contains the
 - ◆ identification of the device,
 - ◆ starting address for the ISR,
 - ◆ address of the branch to the ISR
- ◆ PC finds the ISR address from the code.
- ◆ To add flexibility for multiple devices - corresponding ISR is executed by the processor using a branch address to the appropriate routine - device specified Interrupt Vector.

- ◆ An interrupt vector is the memory address of an interrupt handler, or an index into an array called an interrupt vector table or dispatch table - a table of interrupt vectors (pointers to routines that handle interrupts).
- ◆ Interrupt vector tables contain the memory addresses of interrupt handlers. When an interrupt is generated, the processor saves its execution state via a context switch, and begins execution of the interrupt handler at the interrupt vector.
- ◆ The Interrupt Descriptor Table (IDT) is specific to the I386 architecture. It tells where the Interrupt Service Routines (ISR) are located.
- ◆ Each interrupt number is reserved for a specific purpose. For example, 16 of the vectors are reserved for the 16 IRQ lines.
- ◆ On PCs, the interrupt vector table (IVT or IDT) consists of 256 4-byte pointers - the first 32 (0-31 or 00-1F) of which are reserved for processor exceptions; the rest for hardware interrupts, software interrupts. This resides in the first 1 K of addressable memory.

Interrupt Nesting

- ◆ Pre-Emption of low priority Interrupt by another high priority interrupt is known as Interrupt nesting.
- ◆ Disabling Interrupts during the execution of the ISR may not favor devices which need immediate attention.
- ◆ Need a priority of IRQ devices and accepting IRQ from a high priority device.
- ◆ The priority level of the processor can be changed dynamically.
- ◆ The privileged instruction write in the PS (processor status word), that encodes the processors priority.

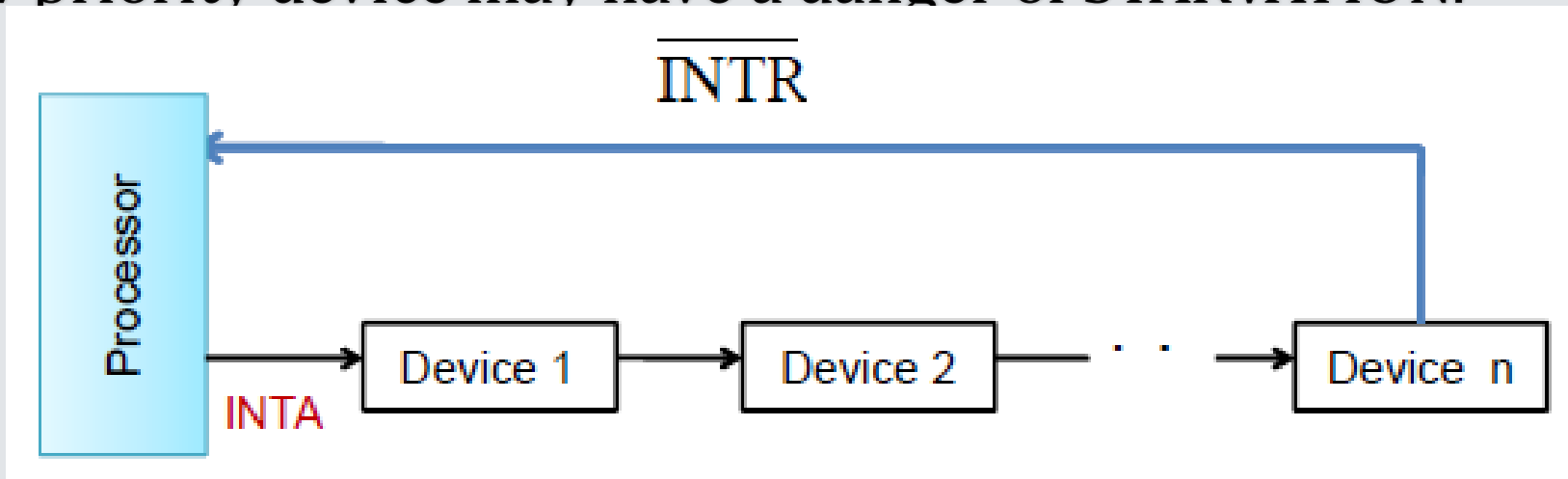
Interrupt Nesting (contd.)



- ◆ Organizing I/O devices in a prioritized structure.
- ◆ Each of the interrupt-request lines is assigned a different priority level.
- ◆ The processor is interrupted only by a high priority device.

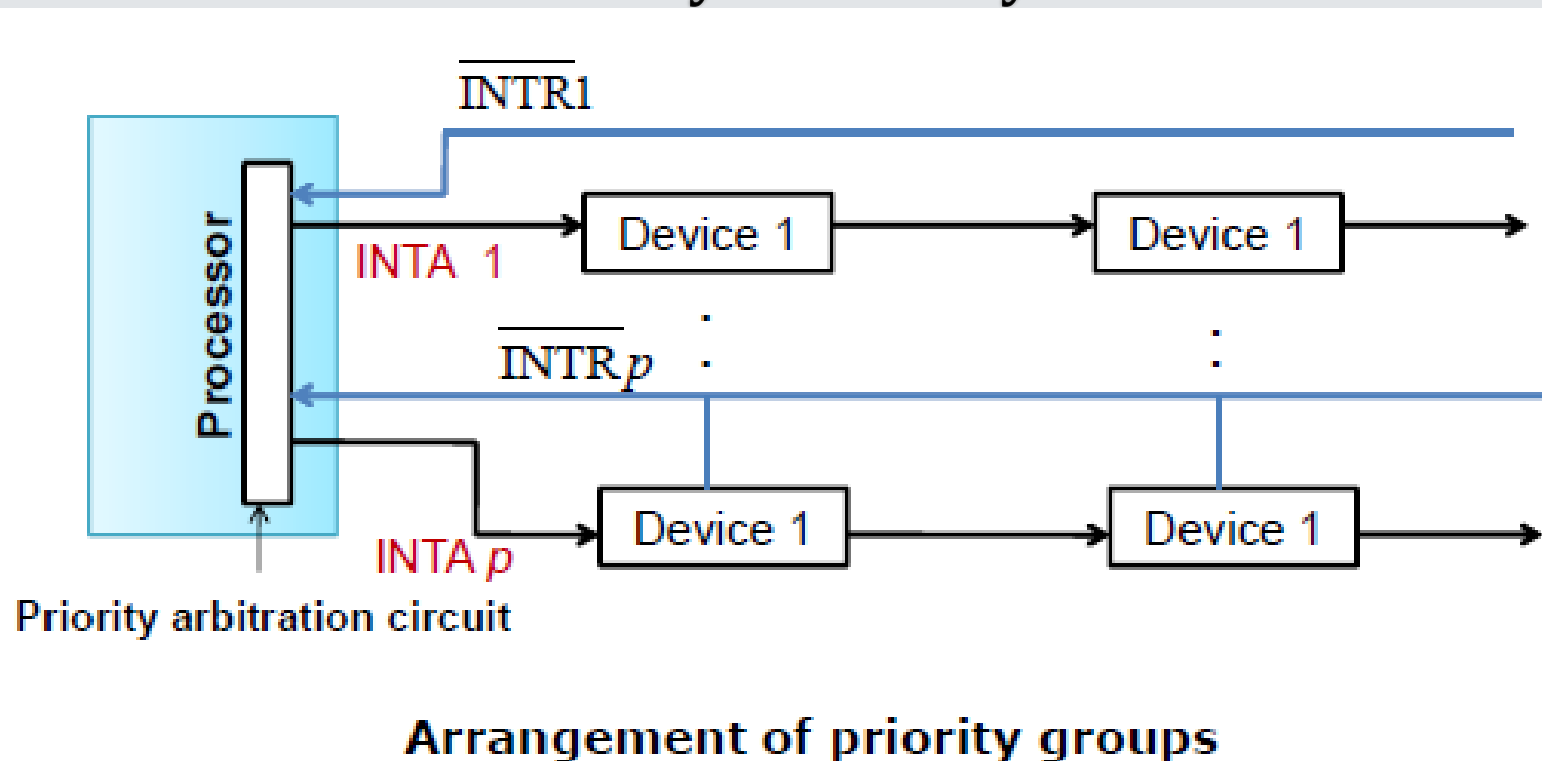
Daisy Chaining

- ◆ The interrupt request line INTR is common to all the devices.
- ◆ The interrupt acknowledgement line INTA is connected to devices in a DAISY CHAIN way.
- ◆ INTA propagates serially through the devices.
- ◆ Device that is electrically closest to the processor gets high priority.
- ◆ Low priority device may have a danger of STARVATION.



Daisy Chaining with Priority Group

- Combining Daisy chaining and Interrupt nesting to form priority group
- Each group has different priority levels and within each group devices are connected in daisy chain way



DMA & Buses



Direct Memory Access

- ◆ For I/O transfer, Processor determines the status of I/O devices, by
 - ◆ Polling
 - ◆ Waiting for Interrupt signal
- ◆ Considerable overhead is incurred in above I/O transfer processing
- ◆ To transfer large blocks of data at high Speed, between EXTERNAL devices & Main Memory, DMA approach is often used
- ◆ DMA controller allows data transfer directly between I/O device and Memory, with minimal intervention of processor.

Direct Memory Access (DMA)

- ❖ DMA controller acts as a Processor, but it is controlled by CPU.
- ❖ To initiate transfer of a block of words, the processor sends the following data to controller.
 - ❖ The starting address of the memory block.
 - ❖ The word count.
 - ❖ Control to specify the mode of transfer such as read or write.
 - ❖ A control to start the DMA transfer.
- ❖ DMA controller performs the requested I/O operation and sends a interrupt to the processor upon completion.

Status and Control

31	30		1	0
IRQ	IE		R/W	Done

Starting address

Word count

In DMA interface

- ◇ First register stores the starting address
- ◇ Second register stores Word count
- ◇ Third register contains status and control flags

Bits and Flags	1	0
R/W	READ	WRITE
Done	Data Transfer Completed	
IRQ	Interrupt Request	
IE	Raise Interrupt(enable) after Data Transfer	

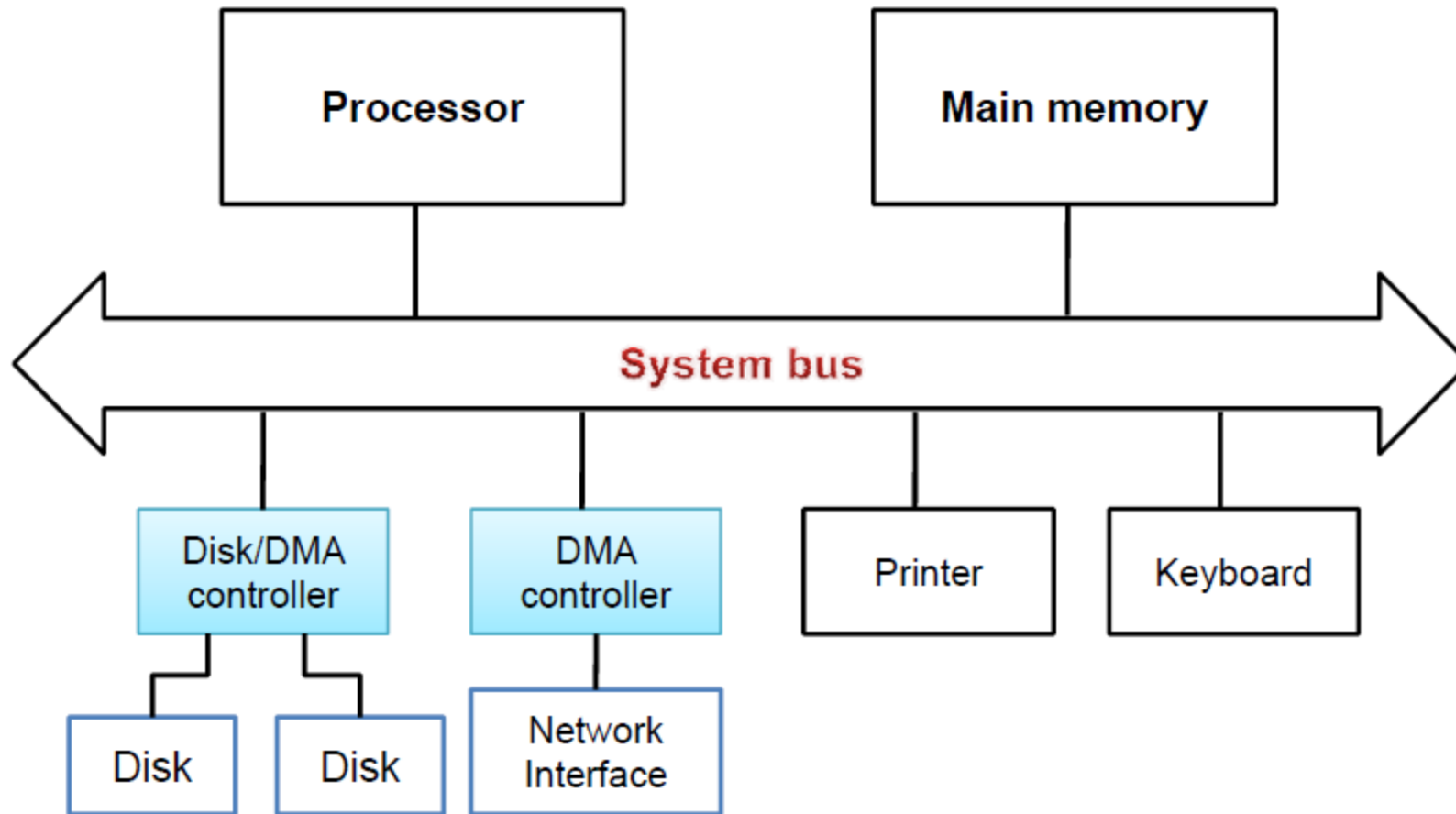
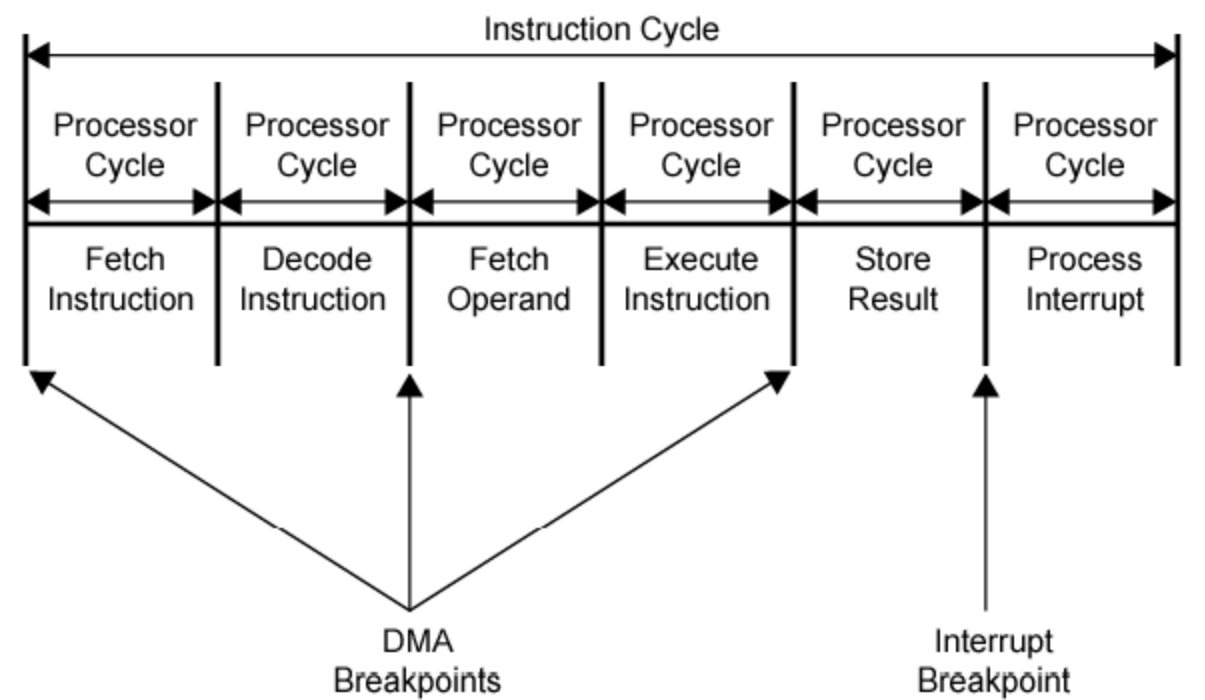


Figure shows the DMA controller in a computer

- ◆ Memory accesses by the processor and DMA Controller are interwoven
- ◆ DMA devices have higher priority than processor over BUS control
- ◆ Cycle Stealing:- DMA Controller “steals” memory cycles from processor, though processor originates most memory access.
- ◆ Block or Burst mode:- The DMA controller may be given exclusive access to the main memory to transfer a block of data without interruption
- ◆ Conflicts in DMA:
 - ◆ Processor and DMA,
 - ◆ Two DMA controllers, try to use the Bus at the same time to access the main memory

DMA and Interrupt Breakpoint



- ❖ Processor does not need to use the bus all of the time (and certainly not for data stored entirely in its registers).
- ❖ This provides opportunities for the DMA controller to transfer data using the bus when it is otherwise idle.
- ❖ The DMA module is able to suspend the processor during its instruction fetch decode-execute cycle.
- ❖ During the DMA breakpoints, the DMA module suspends the processor and transfers a single unit (typically only one byte) between memory and I/O module.
- ❖ Notice that the DMA module does not interrupt the processor, it just suspends it.
- ❖ There is no need for the processor to save its execution context and execute another routine, as the DMA module does not alter this context.
- ❖ The processor runs slower due to its suspension, but not as slow as if it were interrupted at the completion of each byte's transfer or if polling were being used.

BUSES

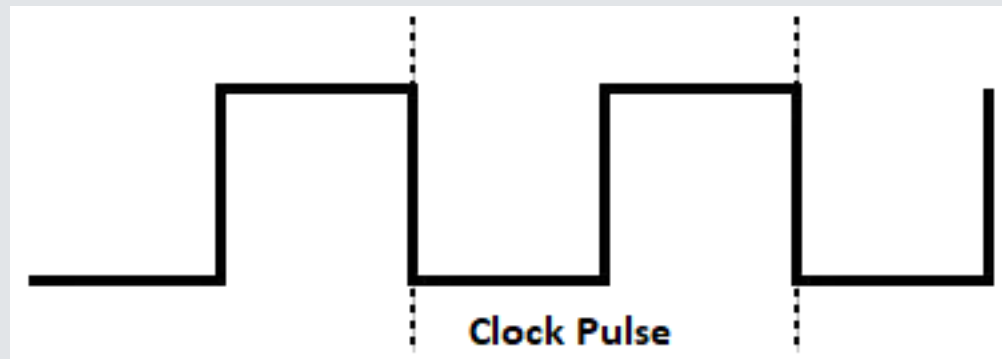
- ◆ Provides interconnection between processor, memory and IO devices acting a communication path to perform data transfer
- ◆ Carries address, data and control signals separately
- ◆ The data size vary from byte, word, long word
- ◆ Control signal like Read/Write as single line, 1 means read & 0 means write
- ◆ Supports to interrupt signal and arbitration
- ◆ Bus carries timing signal

Types of BUSES

- ◆ Vary with timing signal
 - ◆ Synchronous BUS
 - ◆ Asynchronous BUS

Synchronous Bus

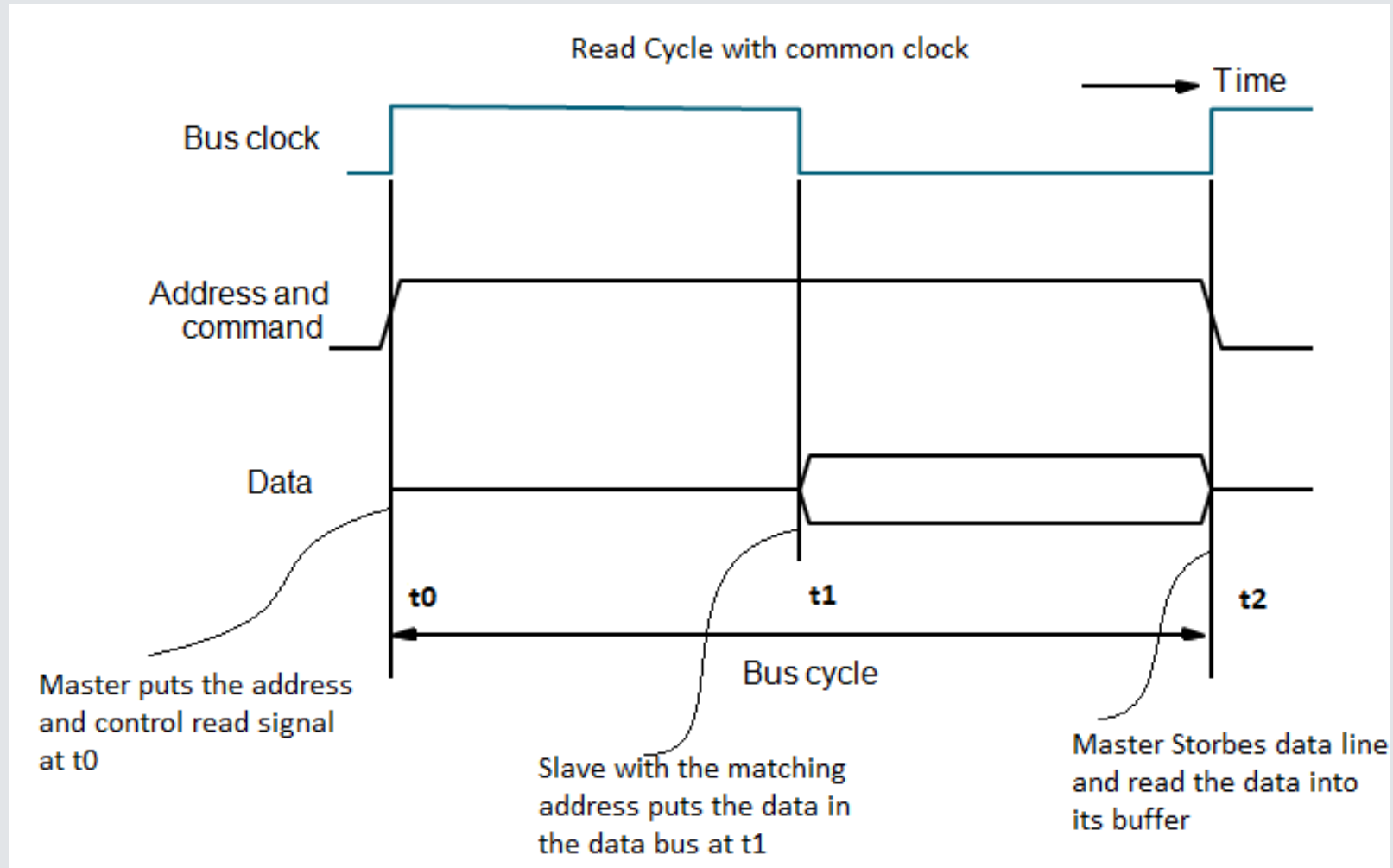
- ◆ Clock signal –generally a square generated using a clock circuitry
- ◆ Every processing devices uses clock pulse to trigger the state change , either at the pulse Edge or level of the square signal



Cont...

- ◊ When all the device uses a common clock then it is said to be synchronous
- ◊ Each pulse in the clock is the bus cycle where the data transfer occurs
- ◊ There will be Master-Slave devices, The device which controls the bus is master and other devices are slave
- ◊ The master signals the address and control signals
- ◊ In read cycle slave put the data over the data lines , master reads the data into its buffer
- ◊ In write cycle master puts the data on bus where the slave reads the data from the bus

Read cycle



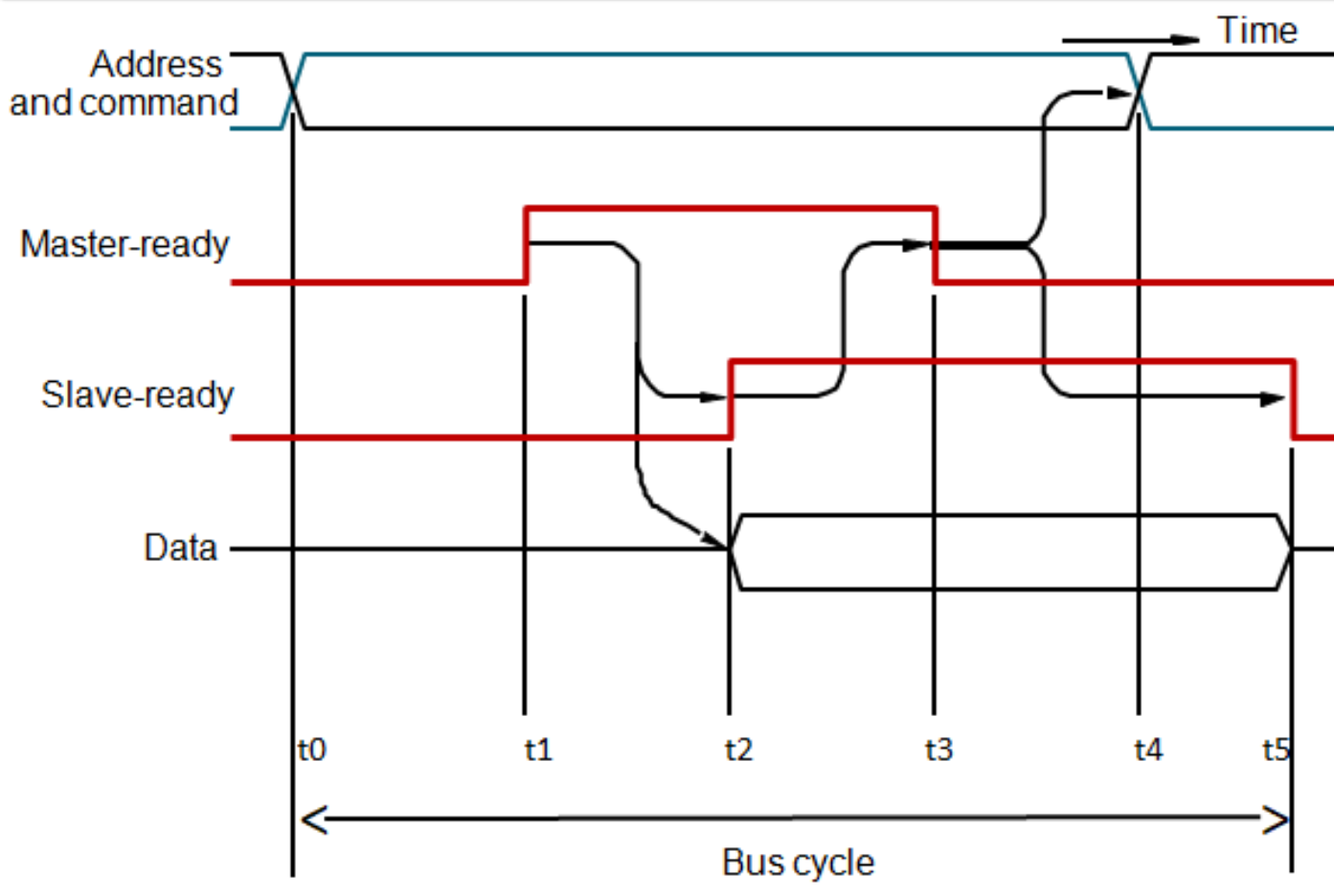
Asynchronous BUS

- ◆ Handshaking signals are used between the master and the slave before the actual data transfer
- ◆ No common clock is used, the master and slave use may two different clocks with varying speed
- ◆ Two control signals to perform handshake
 - ◆ Master-ready (Asserted by the master device to show that the data is ready)
 - ◆ Slave-ready (Slave asserted as a response to indicate that it is ready to transfer data)

Handshake Protocol

- ◆ For read cycle
- ◆ The master puts the address and the control signal on the bus
- ◆ Then it asserts the handshake signal Master-ready on the bus
- ◆ These signals are visible to all the slave devices since they use the same common bus
- ◆ The respective slave puts the data and the signals Slave-ready signal
- ◆ It strobes data from the bus lines into its buffer after the Slave-ready signal

Read cycle using Handshake



t0- Master puts the slave address ,read signal on the bus; which is visible to all the slave devices

t1- Master asserts signal high(Master-ready), to inform the command and address is ready, small skew delay in t0-t1 to match the variance in the speed and propagation delay

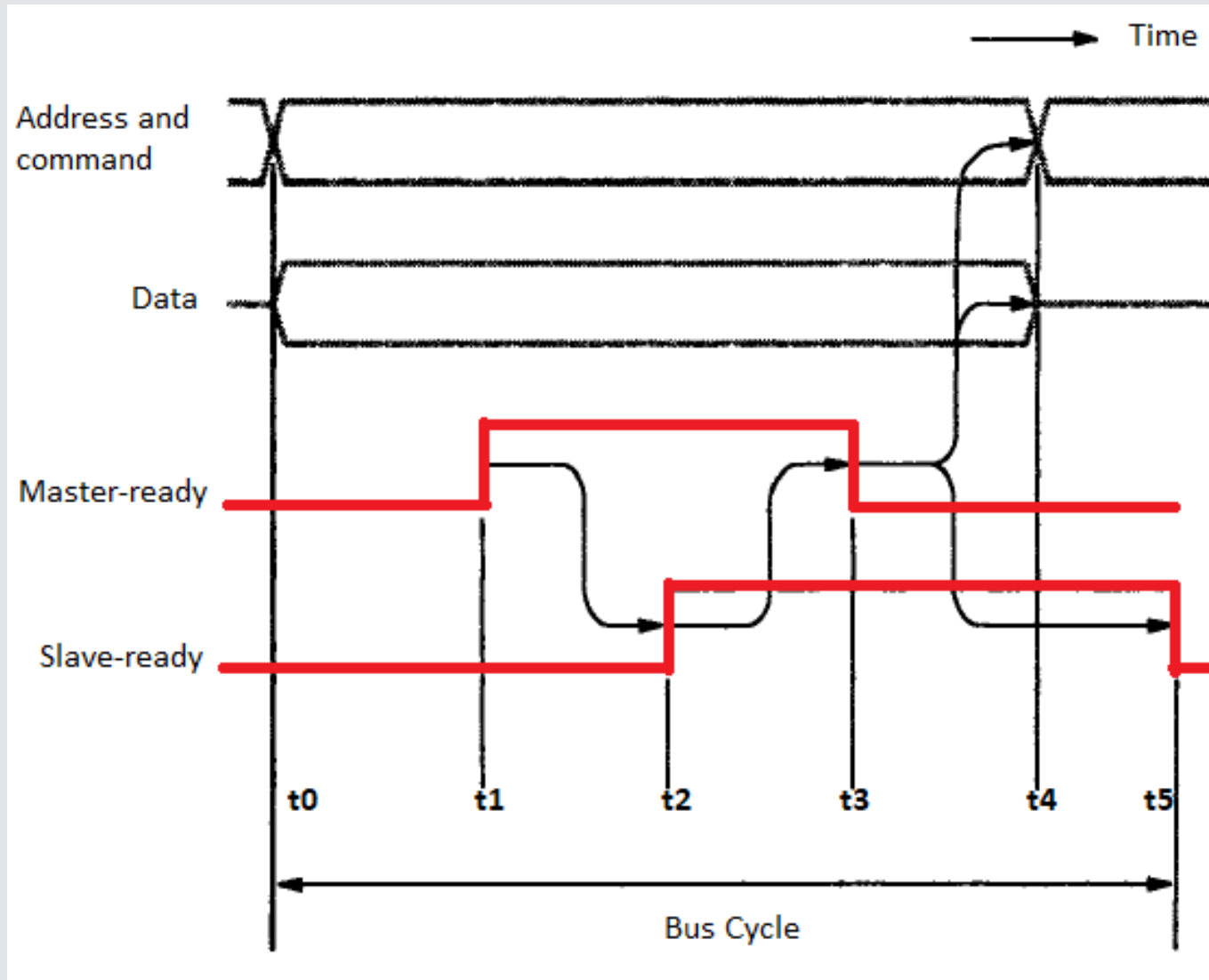
t2 – specified slave decode the address ,command and puts the data on the data bus from its register, by asserting the Slave-ready signal on the bus it informs master

t3-the master receives the handshake signals and strobe the data

t4- Master takes the address and the command off

t5- the slave device removes the data and slave ready signal when it notices the 1 to 0 transition of the master-ready signal

Write Cycle using Handshake





Thank You